

Personalized Weather Intelligence - Project Workflow Document

1. Project Overview

Personalized Weather Intelligence is a FastAPI-based weather intelligence application with a browser-based React frontend. The system combines real-time OpenWeatherMap data, trained machine learning models, and user health profiles to generate personalized weather insights.

The application provides four main outputs:

1. Personalized feels-like temperature
2. Health risk level
3. Activity suitability percentage and safety precautions
4. Forecast-based safer activity time windows
5. Model evaluation dashboard for explaining training results

The frontend is served directly by FastAPI, so the project does not require Node, npm, or Vite to run.

2. Main Technologies

- **Backend:** FastAPI
- **Frontend:** React loaded through browser CDN inside 'frontend/simple/index.html'
- **Weather data:** OpenWeatherMap API
- **Machine learning:** scikit-learn, XGBoost, LightGBM, CatBoost
- **Model persistence:** joblib
- **Data processing:** pandas, numpy
- **External API:** OpenWeatherMap current weather, air pollution, UV and forecast endpoints
- **Runtime interface:** Browser application available at '/ui'

3. Current Project Structure

```
api.py
config.py
requirements.txt
README.md
PROJECT_WORKFLOW.md
frontend/
simple/
index.html

modules/
__init__.py
predictions.py
weather_api.py

models/
train_models.py
feels_like.pkl
```

```
health_risk.pkl
activity.pkl
scaler.pkl
feature_cols.pkl
metrics.json

data/
db_measurements_v2.1.0.csv
db_measurements_v2.1.0.csv.gz
```

4. How To Run The Application

From the project root:

```
cd "G:\Final Year Project\weather_app"
uvicorn api:app --reload --port 8000
```

Then open:

```
http://localhost:8000/ui
```

The API documentation is available at:

```
http://localhost:8000/docs
```

5. Environment Configuration

The application expects an OpenWeatherMap API key in a .env file:

```
OPENWEATHER_API_KEY=your_api_key_here
```

config.py loads this key:

```
API_KEY = os.getenv("OPENWEATHER_API_KEY")
BASE_URL = "http://api.openweathermap.org/data/2.5"
AIR_URL = "http://api.openweathermap.org/data/2.5/air_pollution"
UV_URL = "http://api.openweathermap.org/data/2.5/uvi"
```

6. Dataset Used

The project uses the ASHRAE Global Thermal Comfort Database II:

```
data/db_measurements_v2.1.0.csv
```

The dataset contains thermal comfort measurements such as:

- Air temperature
- Relative humidity
- Air velocity
- Metabolic rate
- Clothing insulation
- Age
- Gender
- Thermal sensation

- PMV
- Operative temperature

The training script selects these columns:

```
cols = [  
    "ta",  
    "rh",  
    "vel",  
    "met",  
    "clo",  
    "age",  
    "gender",  
    "top",  
    "thermal_sensation",  
    "pmv"  
]
```

7. Training Pipeline

Training is handled by:

```
models/train_models.py
```

To retrain models:

```
python models/train_models.py
```

The script:

1. Loads the ASHRAE dataset
2. Cleans missing rows
3. Derives runtime-compatible features
4. Creates target labels
5. Trains multiple algorithms
6. Compares model performance
7. Saves the best models into 'models/'
8. Writes evaluation results into 'models/metrics.json'

8. Runtime Feature Engineering

Runtime feature engineering happens in:

```
modules/predictions.py
```

The runtime feature vector is:

```
feature_temp  
feature_humidity  
feature_wind  
feature_aqi  
feature_age  
feature_health_score  
feature_heat_sens  
feature_cold_sens
```

These features are built from:

- Current weather
- AQI
- User age
- Health conditions
- Heat sensitivity
- Cold sensitivity

The feature order is stored in:

```
FEATURE_COLS = [  
    "feature_temp",  
    "feature_humidity",  
    "feature_wind",  
    "feature_aqi",  
    "feature_age",  
    "feature_health_score",  
    "feature_heat_sens",  
    "feature_cold_sens",  
]
```

At runtime, the application creates a pandas DataFrame with the same feature names used during training. This prevents scikit-learn feature-name warnings.

9. Machine Learning Models

9.1 Feels-Like Temperature Model

Function:

```
predict_feels_like()
```

Model file:

```
models/feels_like.pkl
```

Purpose:

Predicts a personalized perceived temperature.

Input features:

- Temperature
- Humidity
- Wind speed
- AQI
- Age
- Health score
- Heat sensitivity
- Cold sensitivity

The model prediction is then adjusted using a rule-based personalization offset. This offset considers:

- Elderly users
- Heat sensitivity
- Cold sensitivity
- Heart condition
- Asthma
- Respiratory illness
- Diabetes
- Arthritis
- High humidity

Output:

personalized feels-like temperature

9.2 Health Risk Model

Function:

`predict_health_risk()`

Model files:

`models/health_risk.pkl`
`models/scaler.pkl`

Purpose:

Predicts the user's health risk level.

Possible outputs:

Low
 Medium
 High

The system uses a two-layer approach:

1. ML model prediction
2. Rule-based health threshold validation

The rule-based validation checks:

- Heat thresholds
- Cold thresholds
- AQI thresholds
- Humidity thresholds
- UV thresholds
- Age-related vulnerability
- Heart condition
- Asthma
- Respiratory illness
- Diabetes
- Arthritis
- Heat and cold sensitivity

The final risk level is the safer/higher value between ML prediction and rule-based risk.

Returned values:

```
risk_level  
risk_explanation  
confidence  
risk_factors
```

9.3 Activity Suitability Model

Function:

```
predict_activities()
```

Model file:

```
models/activity.pkl
```

Purpose:

Predicts whether selected activities are suitable for the user.

Supported activities:

- Walking
- Running
- Cycling
- Swimming
- Gardening

The system returns:

```
activity status  
suitability percentage  
recommendation text  
limiting factors  
positive factors  
safety precautions
```

Activity status can be:

```
Ideal  
Suitable  
Not Recommended
```

The percentage is generated from the internal activity score.

Example output:

```
{  
  "Running": {  
    "status": "Suitable",  
    "percentage": 61,  
    "recommendation": "61% suitable for running right now.",  
    "limiting_factors": [],  
    "positive_factors": [],  
  },  
}
```

```
"safety_precautions": "To do running more safely..."
}
```

10. Forecast Recommendation System

Forecast data is fetched in:

```
modules/weather_api.py
```

Function:

```
get_forecast(city)
```

It uses OpenWeatherMap's 5-day / 3-hour forecast endpoint:

```
/forecast
```

The application processes the next forecast windows and recommends better times for each selected activity.

Forecast recommendation logic is in:

```
recommend_activity_times()
```

The function checks:

- Forecast temperature
- Forecast humidity
- Forecast wind speed
- Rain probability
- Activity type
- User health conditions
- User age
- Sensitivity profile

For each activity, the system returns the top forecast windows.

Each recommendation includes:

```
datetime
temperature
humidity
wind speed
weather condition
rain probability
suitability percentage
activity note
health and safety note
reason
```

11. Weather API Workflow

Weather functions are located in:

```
modules/weather_api.py
```

Current Weather

Function:

```
get_weather(city)
```

It fetches:

- Temperature
- API feels-like temperature
- Humidity
- Wind speed
- Weather condition
- Description
- Visibility
- Pressure
- Latitude
- Longitude

It also fetches:

- AQI from OpenWeatherMap air pollution API
- UV index where available

Forecast Weather

Function:

```
get_forecast(city)
```

It fetches:

- 3-hour forecast windows
- Temperature
- Feels-like temperature
- Humidity
- Wind speed
- Weather condition
- Rain probability

12. FastAPI Backend Workflow

Main backend file:

```
api.py
```

Main API Endpoint

```
POST /api/predict
```

Request body:

```
{  
  "city": "London",
```

```
"profile": {
  "name": "User",
  "age": 25,
  "gender": "Prefer not to say",
  "health_conditions": ["None"],
  "activity_preferences": ["Walking"],
  "heat_sensitivity": "Medium",
  "cold_sensitivity": "Medium"
}
```

Backend process:

1. Fetch current weather
2. Build profile features
3. Predict personalized feels-like temperature
4. Predict health risk
5. Predict activity suitability
6. Fetch forecast
7. Recommend safer activity windows
8. Return full prediction response

Response includes:

```
weather
feels_like
risk_level
risk_explanation
confidence
risk_factors
activities
forecast
forecast_error
activity_times
```

Other Endpoints

```
GET /ui
```

Serves the React frontend.

```
GET /api/weather/{city}
```

Returns current weather only.

```
GET /api/metrics
```

Returns model evaluation metrics from models/metrics.json.

13. Frontend Workflow

Frontend file:

```
frontend/simple/index.html
```

The frontend uses React through CDN. It is served by FastAPI through:

```
GET /ui
```

Frontend Pages / Steps

The frontend is a step-based single-page application.

Steps:

```
intro
form
predictionLoading
prediction
riskLoading
risk
forecastLoading
forecast
modelLoading
modelEvaluation
```

Step 1: Landing Page

Displays:

- Application name
- Project description
- Key model outputs
- Main factors used by the system
- Start button
- Secondary model evaluation button

Optional Step: Model Evaluation Dashboard

After clicking:

```
View ML Evaluation
```

The frontend calls:

```
GET /api/metrics
```

It displays:

- Deployed model names and headline metrics
- Feels-like model comparison graph
- Activity suitability model comparison graph
- Health-risk model comparison graph
- Feature-importance graph
- Health-risk precision and recall table
- Validation notes for viva/demo explanation

Step 2: Input Form

Collects:

- City
- Name
- Age
- Heat sensitivity
- Cold sensitivity
- Health conditions
- Activity preferences

Step 3: Prediction Loader

Shows animated weather loader while backend prediction is running.

Step 4: Prediction Summary

Displays:

- Temperature
- AQI
- Humidity
- UV level
- Personalized feels-like temperature
- Health risk card
- Activity suitability cards

Step 5: Risk Analysis

After clicking:

Review Health Risk Factors

The frontend shows:

- Risk level
- Confidence
- Summary
- Current weather triggers
- Risk factor cards

Step 6: Forecast Activity Timing

After clicking:

Find Safest Activity Windows

The frontend shows:

- Best forecast time windows
- Activity percentage
- Temperature
- Humidity
- Wind speed
- Rain probability
- Activity check
- Health and safety guidance

14. Full End-To-End Workflow

1. User opens:

`http://localhost:8000/ui`

2. User clicks:

`Start Health Weather Check`

3. User enters city and health profile.

4. Frontend sends request to:

`POST /api/predict`

5. Backend fetches current weather from OpenWeatherMap.

6. Backend fetches AQI and UV data.

7. Backend builds runtime ML features.

8. Backend loads trained ML models from 'models/'.

9. Backend predicts personalized feels-like temperature.

10. Backend predicts health risk level.

11. Backend predicts activity suitability.

12. Backend fetches 3-hour forecast windows.

13. Backend recommends safer activity times.

14. Frontend displays prediction summary.

15. User can open health risk analysis.

16. User can open forecast activity timing recommendations.

17. User can return to the landing page and open the ML evaluation dashboard.

15. Detailed User Journey

This section explains every user-facing step in the current application.

15.1 Landing Page

The landing page introduces the application as Personalized Weather Intelligence. It explains that the system uses weather data, machine learning and health-aware decision support.

The landing page has two actions:

1. ****Start Health Weather Check****
 - This is the main action for a normal user.
 - It opens the profile input form.
2. ****View ML Evaluation****
 - This is a secondary action.

- It opens the evaluation dashboard for academic demonstration, viva discussion and model transparency.

15.2 Profile Input Form

The input form collects the information required for personalisation.

User inputs:

- City
- Name
- Age
- Heat sensitivity
- Cold sensitivity
- Health conditions
- Activity preferences

Health conditions:

- None
- Asthma
- Heart condition
- Diabetes
- Arthritis
- Respiratory illness

Activity preferences:

- Walking
- Running
- Cycling
- Swimming
- Gardening

When the form is submitted, the frontend sends a POST `/api/predict` request to the FastAPI backend.

15.3 Prediction Loading Screen

The loading screen is shown while the backend:

- Fetches live weather
- Fetches air quality information
- Fetches UV information
- Builds the runtime feature vector
- Loads machine learning models
- Generates personalised outputs
- Retrieves forecast windows
- Calculates activity timing recommendations

15.4 Prediction Summary Page

The prediction summary page shows the first set of results.

Displayed cards:

- Current temperature
- Air quality
- Humidity
- UV level
- Personalised feels-like temperature
- Health risk level
- Risk confidence
- Activity suitability cards

Activity cards include:

- Activity name
- Suitability percentage
- Recommendation status
- Main explanation
- Safety precautions

The page then shows a button:

Review Health Risk Factors

15.5 Health Risk Analysis Page

This page explains why the current weather may be safe, cautious or risky for the user.

Displayed information:

- Overall risk level
- Confidence value
- Personal risk explanation
- Current weather triggers
- Risk factor cards

Weather triggers include:

- Temperature
- Humidity
- Air quality
- UV level

Risk factor cards explain which environmental or profile factors affected the result.

The page then shows a button:

Find Safest Activity Windows

15.6 Forecast Activity Timing Page

This page uses forecast data to recommend better times for selected activities.

For each selected activity, the system shows:

- Recommended forecast time
- Suitability percentage
- Temperature
- Humidity

- Wind speed
- Rain probability
- Weather condition
- Activity-specific note
- Health and safety note

The forecast recommendation uses the next available 3-hour forecast windows from OpenWeatherMap.

15.7 Model Evaluation Dashboard

The model evaluation dashboard is designed for academic transparency and viva explanation.

It displays:

- Number of model families compared
- Dataset and runtime data sources
- Selected model for each task
- Feels-like regression metrics
- Activity suitability classification metrics
- Health risk classification metrics
- Model comparison bar graphs
- Feature importance graph
- Per-class health risk precision and recall table
- Explanation of why the Neural Network was selected for health risk

This page calls:

```
GET /api/metrics
```

The metrics are loaded from:

```
models/metrics.json
```

16. Backend Prediction Workflow

The main backend workflow is handled by:

```
api.py
```

The main prediction endpoint is:

```
POST /api/predict
```

Detailed backend sequence:

1. FastAPI receives the city and user profile from the frontend.
2. The backend validates the request body.
3. 'get_weather(city)' is called from 'modules/weather_api.py'.
4. OpenWeatherMap current weather data is requested.
5. Latitude and longitude are extracted from the weather response.
6. Air quality data is requested using the coordinates.
7. UV data is requested using the coordinates.

8. Weather values are normalised into a single Python dictionary.
9. The user profile is converted into runtime features.
10. The trained feels-like model is loaded from 'models/feels_like.pkl'.
11. 'predict_feels_like()' predicts the personalised perceived temperature.
12. A rule-based personalisation offset is applied to account for sensitivity and health profile factors.
13. The trained health risk model is loaded from 'models/health_risk.pkl'.
14. 'predict_health_risk()' predicts the user-specific risk level.
15. Safety thresholds are applied so that health-relevant weather risks are not hidden by the ML model.
16. The trained activity model is loaded from 'models/activity.pkl'.
17. 'predict_activities()' calculates activity suitability percentages.
18. The backend calls 'get_forecast(city)' for future 3-hour forecast windows.
19. 'recommend_activity_times()' ranks future time slots for the user's selected activities.
20. FastAPI returns a JSON response to the frontend.

The response includes:

- Current weather
- Personalised feels-like temperature
- Health risk level
- Risk explanation
- Confidence
- Risk factors
- Activity recommendations
- Forecast information
- Forecast error message if the forecast request fails
- Activity time recommendations

17. Weather API Workflow

Weather data is handled in:

```
modules/weather_api.py
```

The current weather workflow uses these OpenWeatherMap endpoints:

- Current weather endpoint
- Air pollution endpoint
- UV endpoint

The forecast workflow uses:

- 5 day / 3 hour forecast endpoint

The system does not store weather API responses in a database. The response is used immediately for prediction and display.

If weather data cannot be retrieved, the backend returns an error response that is shown on the frontend.

18. Machine Learning Training Workflow

Model training is handled in:

```
models/train_models.py
```

Training sequence:

1. Load the ASHRAE Global Thermal Comfort Database II.
2. Select relevant thermal comfort columns.
3. Clean rows with missing values.
4. Build runtime-compatible features.
5. Create targets for feels-like prediction, activity suitability and health risk.
6. Split data into training and testing sets.
7. Train multiple algorithms for each task.
8. Compare models using appropriate metrics.
9. Select deployment models.
10. Save trained models into the 'models/' directory.
11. Save feature column order.
12. Save scaler for the neural network path.
13. Save evaluation metrics into 'models/metrics.json'.

Model artifacts:

```
models/feels_like.pkl  
models/activity.pkl  
models/health_risk.pkl  
models/scaler.pkl  
models/feature_cols.pkl  
models/metrics.json
```

19. Model Evaluation Results

19.1 Feels-Like Temperature Model

Task type:

Regression

Selected model:

Random Forest Regressor

Metrics:

- MAE: 0.216
- RMSE: 0.367
- Cross-validation MAE mean: 0.314
- Cross-validation MAE standard deviation: 0.029

Compared algorithms:

- Linear Regression
- Decision Tree
- Random Forest
- Gradient Boosting

- XGBoost
- CatBoost

Interpretation:

The Random Forest model achieved the lowest error among the tested regression models. Feature importance shows that temperature dominates the prediction. This is expected for perceived temperature modelling, but it also means that personal profile variables have lower influence in the trained model. The system addresses this limitation by applying a rule-based personalisation offset after the ML prediction.

19.2 Activity Suitability Model

Task type:

Classification and recommendation scoring

Selected model:

Random Forest Classifier

Metrics:

- F1: 0.968
- Accuracy: 0.965
- Precision: 0.962
- Recall: 0.965
- Cross-validation F1 mean: 0.950
- Cross-validation F1 standard deviation: 0.009

Compared algorithms:

- Decision Tree
- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost

Interpretation:

Weighted F1 is important because the system should balance correct suitable and unsuitable recommendations. The activity model is supported by additional rule-based checks for temperature, AQI, humidity, UV, wind and health conditions.

19.3 Health Risk Model

Task type:

Classification

Selected model:

Neural Network, MLP

Metrics:

- Accuracy: 0.992
- F1: 0.992
- Precision: 0.992
- Recall: 0.992

Compared algorithms:

- Logistic Regression
- Decision Tree
- Random Forest
- Gradient Boosting
- XGBoost
- LightGBM
- CatBoost
- Neural Network, MLP

Important explanation:

Random Forest achieved a slightly higher raw health-risk score, around 0.995 accuracy. The Neural Network was selected because it remained within approximately 1 percent of the best raw result and is conceptually suitable for modelling non-linear interactions between environmental factors, age, sensitivity and health conditions.

Per-class health risk results:

```
Low risk:    precision 0.994, recall 1.000
Medium risk: precision 0.987, recall 0.979
High risk:   precision 0.958, recall 0.719
```

Critical limitation:

The high-risk recall of 0.719 is lower than the low and medium risk classes. This matters because a safety-related system should avoid missing high-risk cases. The project partly addresses this by combining ML predictions with rule-based health and weather thresholds. Future work should include more high-risk data, class balancing, calibration, external validation and expert review.

20. Runtime Feature Engineering

The runtime feature vector is built in:

```
modules/predictions.py
```

Features:

```
feature_temp
feature_humidity
feature_wind
feature_aqi
feature_age
feature_health_score
feature_heat_sens
feature_cold_sens
```

Feature sources:

- Temperature comes from OpenWeatherMap.
- Humidity comes from OpenWeatherMap.
- Wind speed comes from OpenWeatherMap.
- AQI comes from OpenWeatherMap air pollution data.
- Age comes from the user profile.
- Health score is derived from selected health conditions.
- Heat sensitivity is encoded from user selection.
- Cold sensitivity is encoded from user selection.

The feature vector is created as a pandas DataFrame using the same feature names used during model training. This prevents scikit-learn feature-name warnings.

21. Hybrid Decision Logic

The project does not rely only on raw machine learning outputs. It uses a hybrid approach:

1. Machine learning model prediction
2. Rule-based personalisation
3. Safety threshold validation
4. Human-readable explanation

This approach is used because weather-health advice has safety implications. A pure model output could miss a risk if the training data is limited. The rule-based layer helps ensure that known risk conditions, such as very poor air quality or high sensitivity, are still considered.

22. Frontend Implementation Details

Frontend file:

```
frontend/simple/index.html
```

The frontend is a single page React application. It is served by FastAPI and does not require a separate frontend server.

Main React states:

```
intro
form
predictionLoading
prediction
riskLoading
risk
forecastLoading
forecast
modelLoading
modelEvaluation
```

Important frontend features:

- Professional landing page
- Editable name and age fields
- Multi-select health conditions

- Multi-select activity preferences
- Animated loaders
- Prediction metric cards
- Risk factor cards
- Forecast activity cards
- Model evaluation dashboard
- Back navigation between major pages

The current frontend intentionally does not include search history because the project no longer uses a database.

23. Removed Or Unused Functionality

The project previously included or considered database-backed profile/history functionality. This was removed from the current version.

Reasons for removal:

- Search history was not central to the final project aim.
- The current dissertation focus is weather intelligence, ML prediction and health-aware activity recommendation.
- Removing the database simplifies installation and demonstration.
- The current app avoids storing personal health profile data persistently.

Removed or no longer used:

- Database storage
- Search history page
- Profile persistence endpoints
- Node/Vite frontend workflow

Current project does not require:

- npm
- Node.js
- A database server
- A local frontend dev server

24. Testing And Validation Checklist

Suggested checks for demonstration:

1. Start FastAPI using 'uvicorn api:app --reload --port 8000'.
2. Open 'http://localhost:8000/ui'.
3. Confirm landing page loads.
4. Click **Start Health Weather Check**.
5. Enter a city and profile.
6. Submit the form.
7. Confirm prediction summary appears.
8. Confirm temperature, AQI, humidity and UV cards show values.
9. Confirm personalised feels-like temperature appears.
10. Confirm health risk card appears.
11. Confirm activity suitability cards show percentages and safety advice.

12. Click ****Review Health Risk Factors****.
13. Confirm risk analysis page appears.
14. Confirm risk factor cards and weather triggers are displayed.
15. Click ****Find Safest Activity Windows****.
16. Confirm forecast timing cards appear.
17. Return to landing page.
18. Click ****View ML Evaluation****.
19. Confirm model evaluation graphs and tables appear.
20. Open 'http://localhost:8000/docs' and confirm API documentation is available.
21. Open 'http://localhost:8000/api/metrics' and confirm model metrics are returned.

25. Where AI / ML Is Used

AI/ML is used in:

```
modules/predictions.py
```

The model files are:

```
models/feels_like.pkl  
models/health_risk.pkl  
models/activity.pkl
```

ML outputs:

```
personalized feels-like temperature  
health risk level  
activity suitability
```

Rule-based logic is also used for safety validation and explanation.

This means the system is not only a weather display app. It is a weather intelligence system that applies trained models and health-aware reasoning to produce personalized recommendations.

26. Important Notes And Limitations

- The ASHRAE dataset is mainly based on thermal comfort data, often indoor or controlled-environment focused.
- Applying this to outdoor weather is useful for a prototype but should be discussed as a limitation.
- Health recommendations are decision-support only and should not be treated as medical diagnosis.
- The forecast timing system depends on OpenWeatherMap forecast availability.
- UV data availability may vary depending on the API endpoint and subscription support.
- The frontend uses React through browser CDN, so internet access is needed for first page load.
- The high-risk class has lower recall than the other risk classes and should be discussed critically.
- The current application does not persist user profiles or history.
- The model evaluation page shows saved training metrics and should not be

interpreted as clinical validation.

27. Suggested Explanation For Presentation

This project addresses the limitation that most weather applications show the same weather values to every user. In this system, the same temperature can produce different recommendations depending on age, health condition, sensitivity, and selected activities. The backend combines real-time weather data with trained machine learning models and rule-based health validation to produce personalized weather intelligence.

The system demonstrates:

- Data collection through weather APIs
- Machine learning model training
- Model deployment through FastAPI
- User profile personalization
- Health-aware risk analysis
- Forecast-based activity planning
- Browser-based React frontend
- Model evaluation and transparency dashboard

28. Key Files To Give Another Developer Or Chatbot

If another person or chatbot needs to understand the project, provide these files:

```
PROJECT_WORKFLOW.md
api.py
modules/weather_api.py
modules/predictions.py
models/train_models.py
models/metrics.json
frontend/simple/index.html
requirements.txt
README.md
```

29. Short Summary

Personalized Weather Intelligence is a FastAPI and React application that uses OpenWeatherMap data, trained ML models, health-profile rules, and forecast analysis to produce personalized weather, health risk, activity suitability, and safer activity timing recommendations.